

Sparse Table :

```

/*
SparseTable sp(a, [](int a, int b) {
    return max(a, b);
});
*/
template<typename T, class CMP = function<T(const T &, const T &)>>
class SparseTable {
public:
    int n;
    vector<vector<T>> sp;
    CMP func;
    SparseTable(const vector<T> &a, const CMP &f) : func(f) {
        n = a.size();
        int max_log = 32 - __builtin_clz(n);
        sp.resize(max_log);
        sp[0] = a;
        for (int j = 1; j < max_log; ++j) {
            sp[j].resize(n - (1 << j) + 1);
            for (int i = 0; i + (1 << j) <= n; ++i) {
                sp[j][i] = func(
                    sp[j - 1][i],
                    sp[j - 1][i + (1 << (j - 1))])
            };
        }
    }
    T get(int l, int r) const {
        int lg = __lg(r - l + 1);
        return func(
            sp[lg][l],
            sp[lg][r - (1 << lg) + 1]
        );
    }
};

```

Build Segment Tree :

```

void build (int l , int r , int node , vector<int> & arr){
    if(l==r){
        if(l<arr.size()){
            seg[node] = arr[l];
        }
        return;
    }
}

```

```

    }
    build(l,mid,L,arr);
    build(mid+1,r,R,arr);
    seg[node] = merge (seg[L],seg[R]);
}

```

Segment Tree :

```

struct Segtree
{
    #define L 2*node+1
    #define R 2*node+2
    #define mid ((l+r)/2)

    vector<int> seg ;
    int sz, skip ;

    int merge (int a ,int b){
        return a+b ;
    }

    int query (int l , int r , int node ,int lx , int rx){
        if(r<lx || rx<l)return skip ;
        if(l>=lx&&r<=rx) return seg[node];
        int left = query (l , mid ,L , lx , rx);
        int right= query (mid+1,r,R , lx,rx);
        return merge(left,right);
    }

    void update (int l , int r, int node , int indx , int val){
        if(l==r){
            seg[node]=val;
            return ;
        }
        if(indx<=mid){
            update(l,mid,L,indx,val);
        }
        else{
            update(mid+1,r,R,indx,val);
        }
        seg[node] = merge (seg[L],seg[R]);
    }

    Segtree(int szz){
        int n = szz;
        sz = 1 ;
        skip = 0 ;
        while (sz<n) sz*=2 ;
        seg = vector<int> (sz*2 , skip);
    }
}

```

```

int queryCall(int l , int r){
    return query(0,sz-1,0,l,r);
}
void updateCall(int val , int indx){
    update(0,sz-1,0,indx,val);
}

#undef L
#undef R
#undef mid
};

```

Lazy Segment Tree :

```

struct Segtree
{
#define L 2*node+1
#define R 2*node+2
#define mid ((l+r)/2)

vector<int> seg , typ , lazy;
int sz, skip ;

int merge (int a ,int b){
    return a+b ;
}

void propagate (int l , int r , int node){
    if(typ[node]==-1)return ;
    if(l!=r){
        lazy[L] += lazy[node];
        lazy[R] += lazy[node];
        typ[L] = typ[node] ;
        typ[R] = typ[node] ;
    }
    seg[node] += lazy[node]*(r-l+1);
    lazy[node] = 0 ;
    typ[node] = -1 ;
}

int query (int l , int r , int node ,int lx , int rx){
    propagate(l,r,node);
    if(r<lx || rx<l)return skip ;
    if(l>=lx&&r<=rx) return seg[node];
    int left = query (l , mid ,L , lx , rx);
    int right= query (mid+1,r,R , lx,rx);
    return merge(left,right);
}

void update (int l , int r , int node , int lx , int rx, int val){
    propagate(l,r,node);
    if(r<lx || rx<l)return ;

```

```

    if(l>=lx&&r<=rx) {
        lazy [node] += val ;
        typ [node] = 1;
        propagate(l,r,node);
        return ;
    }
    update(l,mid,L,lx,rx,val);
    update(mid+1,r,R,lx,rx,val);
    seg[node] = merge (seg[L],seg[R]);
}

Segtree(int sz){
    int n = sz;
    sz = 1 ;
    skip = 0 ;
    while (sz<n) sz*=2 ;
    seg = vector<int> (sz*2 , skip);
    lazy = vector<int>(sz*2 , skip) ;
    typ = vector<int>(sz*2 , -1) ;
}

int queryCall(int l , int r){
    return query(0,sz-1,0,l,r);
}

void updateCall(int l , int r , int val){
    update(0,sz-1,0,l,r,val);
}

#undef L
#undef R
#undef mid
};

```

BIT Point Update Range Query :

```

struct BIT {
    int n;
    vector<int> bit;

    BIT(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void add(int i, int val) {
        ++i;
        while (i <= n) {
            bit[i] += val;
            i += i & -i;
        }
    }
};

```

```

    }
}

int prefixQuery(int i) {
    ++i;
    int res = 0;
    while (i > 0) {
        res += bit[i];
        i -= i & -i;
    }
    return res;
}

int rangeQuery(int l, int r) {
    return prefixQuery(r) - prefixQuery(l - 1);
}
};

```

BIT Range Update Point Query :

```

struct BIT {
    int n;
    vector<int> bit;

    BIT(int _n) {
        n = _n;
        bit.assign(n + 1, 0); // size is n+1 to allow access up to index n-1
    }

    // Add value to position i
    void add(int i, int val) {
        ++i; // convert to 1-based indexing internally
        while (i <= n) {
            bit[i] += val;
            i += i & -i;
        }
    }

    // Query the prefix sum in range [0, i]
    int prefixQuery(int i) {
        ++i;
        int res = 0;
        while (i > 0) {
            res += bit[i];
            i -= i & -i;
        }
        return res;
    }

    // Get value at a single index (for point update + range query version)

```

```

int pointQuery(int i) {
    return prefixQuery(i);
}
// Perform range update: add `val` to all elements in [l, r]
void rangeAdd(int l, int r, int val) {
    add(l, val);
    add(r + 1, -val);
}
};

```

MO Algo :

```

void solve(){
    int n , q ; cin>>n>>q ;
    int arr[n] ;
    vector<array<int,3>>q;
    vector<int>res(q) ;
    vector<int>freq(2e5+5);
    int id = 1 ;
    int BS = sqrt(n) ;

    lp(i,n)cin>>arr[i];

    lp(i,q){
        int l , r ; cin>>l>>r ;
        qi.push_back({--l,--r,i});
    }
    map<int,int>mp ;
    for (int i = 0; i < n; i++)
    {
        if(!mp[arr[i]]){
            mp[arr[i]]=id; arr[i]= id ; id++;
        }else{
            arr[i] = mp[arr[i]] ;
        }
    }

    int ans = 0 ;
    auto add = [&](int index){
        int val = arr[index] ;
        if(!freq[val])ans++ ;
        freq[val]++;
    };
    auto erase = [&](int index){
        int val = arr[index] ;
        freq[val]--;
        if(!freq[val])ans-- ;
    };
};

```

```

sort(qi.begin(),qi.end(),[&](array<int,3>q1 , array<int,3>q2){
    return make_pair(q1[0]/BS , q1[1]) < make_pair(q2[0]/BS , q2[1]) ;
});
int l = qi[0][0] ;
int r = l ;
add(l);
for(auto [lx , rx , ind]: qi){
    while (lx<l)add(--l);
    while (rx>r)add(++r);

    while (lx>l)erase(l++);
    while (rx<r)erase(r--);

    res[ind] = ans;
}
for (int i = 0; i < q; i++)cout<<res[i] ln ;
}

```

MO with RollBack

```

void solve(){
    int n , m , k ; cin>>n>>m>>k ;
    int arr[n] ;
    lp(i,n)cin>>arr[i] ;
    int B = sqrt(n)+2 ;
    vector<array<int,3>>qi [B] ;
    map<int,int>res ;
    m++;
    int tL[m] ;
    memset(tL , -1 , sizeof tL) ;
    for (int i = 0; i < k; i++)
    {
        int l , r ; cin>>l>>r ;
        l-- , r--;
        if(r-l+1<=B){
            int a = 0 ;
            for (int j = l; j <= r; j++)
            {
                int val = arr[j] ;
                if(tL[val]==-1)tL[val] = j ;
                else a = max(a,abs(j-tL[val]));
            }
            res[i] = a ;
            for (int j = l; j <= r; j++)
            {
                int val = arr[j] ;
                tL[val] = -1 ;
            }
        }
    }
}

```

```

    }else{
        qi[l/B].push_back({r,l,i});
    }
}
for (int i = 0; i < B; i++)
{
    sort(qi[i].begin(),qi[i].end());
}
int L[m] , R[m] ;

for (int i = 0; i < B; i++)
{
    int ans = 0 ;
    if(qi[i].empty())continue;
    for (int i = 0; i < m; i++)L[i] = R[i] = -1 ;
    int l = B*i+(B-1) ;
    int lst = l ;
    int r = l ;
    L[arr[l]] = R[arr[l]] = 1 ;
    for (auto [rq , lq, iq ] : qi[i] )
    {
        while (rq>r)
        {
            r++;
            int val = arr[r] ;
            if(R[val]==-1){
                L[val] = R[val] = r ;
            }else{
                R[val] = r ;
                ans = max(ans , abs(L[val]-R[val]));
            }
        }
        vector<array<int,3>>updates ;
        int tmp = ans ;
        while (l>lq)
        {
            l--;
            int val = arr[l] ;
            updates.push_back({L[val],R[val],val});
            if(R[val]==-1){
                L[val] = R[val] = l ;
            }else{
                L[val] = l ;
                tmp = max(tmp , abs(L[val]-R[val]));
            }
        }
        res[iq] = max(tmp,ans);
        reverse(updates.begin(),updates.end());
        for(auto u : updates){
            L[u[2]] = u[0];
            R[u[2]] = u[1];
        }
        l = lst ;
    }
}

```



```

    }
    for (int i = 0; i < k; i++)
    {
        cout<<res[i] ln ;
    }
}

```

MO with DSU :

```

void solve(){
    int n , m ; cin>>n>>m ;
    vp arr(m) ;

    int B = sqrt(m)+2;
    lp(i,m){
        int a,b ;cin>>a>>b; a--;b--;
        arr[i].first = a ; arr[i].second = b;
    }
    int q ; cin>>q ;
    vi res(q) ;
    vector<array<int,3>>qi [B];
    DSU d ; d.init(n) ; d.snapshot();
    for (int i = 0; i < q; i++)
    {
        int a , b ; cin>>a>>b; a--; b--;
        if(b-a+1<=B){
            d.snapshot();
            for (int j = a; j <= b; j++)
            {
                d.merge(arr[j].first,arr[j].second);
            }
            res[i] = d.cnt ;
            d.rollback();
            continue;
        }
        qi[a/B].push_back({b,a,i});
    }
    for (int i = 0; i < B; i++)
    {
        sort(all(qi[i]));
    }
    for (int i = 0; i < B; i++)
    {
        if(qi[i].empty())continue;
        int l = min(B*i+(B-1),m-1) ;
        int r = l ;
        int tempL = l ;
        d.snapshot();
        d.merge(arr[l].first,arr[l].second);
    }
}

```

```

for(auto [rx , lx , qx] : qi[i]){
    while (rx>r)
    {
        r++; d.merge(arr[r].first , arr[r].second);
    }
    d.snapshot();
    while (lx<l)
    {
        l--; d.merge(arr[l].first , arr[l].second);
    }
    res[qx] = d.cnt ;
    d.rollback();
    l = tempL ;
}
d.rollback();
}
for (int i = 0; i < q; i++)cout<<res[i] ln ;
}

```

Orderd Set :

Definition :

```

/////
#include <ext/pb_ds/assoc_container.hpp> // Common file
#include <ext/pb_ds/tree_policy.hpp> // Including
tree_order_statistics_node_update
using namespace __gnu_pbds;
#define ordered_set tree<int, null_type,less<int>,
rb_tree_tag,tree_order_statistics_node_update>
// for multiset -> less_equel but erase will not work
// for count numbers >= x -> order_of_key(x+1)
/////

```

How to use :

define an object like `this : ordered_set st ;`
then you can deal with it as you deal with set

Extra Functions :

`find_by_order(k)` .. $O(\log(N))$

It returns an iterator to the kth element (counting from zero) in the set in $O(\log(n))$ time.

To find the first element k must be zero.

Let us assume we have a set `s : {1, 5, 6, 17, 88}`,

then :

`*(s.find_by_order(2))` : 3rd element in the set is 6

`*(s.find_by_order(4))` : 5th element in the set is 88

```
order_of_key(k).. O(log(N))//
```

It returns to the number of items that are strictly smaller than our item k in $O(\log(n))$ time.

Let us assume we have a set s : {1, 5, 6, 17, 88}

then :

s.order_of_key(6) : Count of elements strictly smaller than 6 is 2.

s.order_of_key(25) : Count of elements strictly smaller than 25 is 4.

How to change it to Ordered Multiset :

less --> less_equal

but if you want to erase an element .. you must erase it like this :

```
st.erase(st.upper_bound(X));
```

Segment tree with hashing :

```
const int N = 1e6 + 5;
const int MODS[] = {1000000000 + 7, 1000000000 + 9, 1000000000 + 21, 1000000000 + 33, 1000000000 + 87};
mt19937_64 rng(chrono::system_clock::now().time_since_epoch().count());
int MOD = MODS[uniform_int_distribution<int>(0, 4)(rng)];
int BASE = uniform_int_distribution<int>(256, MOD - 2)(rng);
int pw[N];
void build() {
    pw[0] = 1;
    for (int i = 1; i < N; i++)
        pw[i] = 1LL * pw[i - 1] * BASE % MOD;
}

struct Node
{
    int sz , pref , rev ;
};
struct Segtree
{
#define L 2*node+1
#define R 2*node+2
#define mid ((l+r)/2)

vector<Node> seg ;
int sz; Node skip ;

Node merge (Node a ,Node b){
    Node res ;

    res.sz = a.sz + b.sz ;
    res.pref = (a.pref*pw[b.sz] + b.pref)%MOD;
    res.rev = (b.rev*pw[a.sz] + a.rev)%MOD;

    return res ;
}
```

```

}
Node query (int l , int r , int node ,int lx , int rx){
    if(r<lx || rx<l)return skip ;
    if(l>=lx&&r<=rx) return seg[node];
    Node left = query (l , mid ,L , lx , rx);
    Node right= query (mid+1,r,R , lx,rx);
    return merge(left,right);
}
void update (int l , int r, int node , int indx , int val){
    if(l==r){
        seg[node].pref = val;
        seg[node].rev  = val;
        seg[node].sz = 1;
        return ;
    }
    if(indx<=mid){
        update(l,mid,L,indx,val);
    }
    else{
        update(mid+1,r,R,indx,val);
    }
    seg[node] = merge (seg[L],seg[R]);
}

Segtree(int sz){
    int n = sz;
    sz = 1 ;

    skip.pref = 0 ;
    skip.rev = 0 ;
    skip.sz = 0 ;
    while (sz<n) sz*=2 ;
    seg = vector<Node> (sz*2 , skip);
}

bool queryCall(int l , int r){
    Node res = query(0,sz-1,0,l,r);
    return res.pref == res.rev ;
}
void updateCall(int val , int indx){
    update(0,sz-1,0,indx,val);
}

#undef L
#undef R
#undef mid
};

```

Lazy Segment Tree With Hashing

```

const int N = 1e6 + 5;
const int MODS[] = {1000000000 + 7, 1000000000 + 9, 1000000000 + 21, 1000000000 + 33, 1000000000 + 87};
mt19937_64 rng(chrono::system_clock::now().time_since_epoch().count());
int MOD = MODS[uniform_int_distribution<int>(0, 4)(rng)];
int BASE = uniform_int_distribution<int>(256, MOD - 2)(rng);
int pw[N] , sumPw[N];
void build() {
    pw[0] = 1; sumPw[0] = 1;
    for (int i = 1; i < N; i++){
        sumPw[i] = (1LL * sumPw[i - 1] * BASE + 1) % MOD;
        pw[i] = 1LL * pw[i - 1] * BASE % MOD;
    }
}

struct Node
{
    int sz , pref , rev ;
};
struct Segtree
{
    #define L 2*node+1
    #define R 2*node+2
    #define mid ((l+r)/2)

    vector<Node> seg ;
    vector<int> typ , lazy ;
    int sz; Node skip ;

    Node merge (Node a ,Node b){
        Node res ;
        res.sz = a.sz + b.sz ;
        res.pref = (1LL*a.pref*pw[b.sz] + b.pref)%MOD;
        res.rev = (1LL*b.rev *pw[a.sz] + a.rev)%MOD;
        return res ;
    }

    void apply(int node, int val) {
        seg[node].pref = 1LL * val * sumPw[seg[node].sz - 1] % MOD;
        seg[node].rev = seg[node].pref;
        lazy[node] = val;
        typ[node] = 1;
    }

    void propagate(int l, int r, int node) {
        if (typ[node] == -1) return;

        if (l != r) {
            apply(L, lazy[node]);
            apply(R, lazy[node]);
        }

        lazy[node] = 0;
        typ[node] = -1;
    }
}

```

```

}
Node query (int l , int r , int node ,int lx , int rx){
    propagate(l,r,node);
    if(r<lx || rx<l)return skip ;
    if(l>=lx&&r<=rx) return seg[node];
    Node left = query (l , mid ,L , lx , rx);
    Node right= query (mid+1,r,R , lx,rx);
    return merge(left,right);
}

void update (int l , int r, int node , int lx , int rx, int val){
    propagate(l,r,node);
    if(r<lx || rx<l)return ;
    if(l>=lx&&r<=rx) {
        apply(node, val);
        return ;
    }
    update(l,mid,L,lx,rx,val);
    update(mid+1,r,R,lx,rx,val);
    seg[node] = merge (seg[L],seg[R]);
}

void buildTree(int l,int r,int node){
    seg[node].sz = r-l+1;
    if(l==r) return;
    buildTree(l,mid,L);
    buildTree(mid+1,r,R);
}

Segtree(int sz){
    int n = sz;
    sz = 1 ;
    skip.pref = 0 ;
    skip.rev = 0 ;
    skip.sz = 0 ;
    while (sz<n) sz*=2 ;
    seg = vector<Node> (sz*2 , skip);
    lazy = vector<int>(sz*2 , 0) ;
    typ = vector<int>(sz*2 , -1) ;
}

bool queryCall(int l , int r){
    Node res = query(0,sz-1,0,l,r);
    return res.pref == res.rev ;
}

void updateCall(int l, int r, int val){
    update(0, sz-1, 0, l, r, val);
}

#undef L
#undef R
#undef mid
};

```

```
// Segtree t (s.size());
// t.buildTree(0 , t.sz-1 , 0);
```

Segment Tree for Max Freq in Range

```
struct Node {
    int val, freq;
    int prefixVal, prefixFreq;
    int suffixVal, suffixFreq;
};

struct SegmentTree {
    #define L 2 * node + 1
    #define R 2 * node + 2
    #define mid ((l + r) / 2)
    int sz;
    vector<Node> tree;
    vector<int> arr;
    Node skip = {-1, 0, -1, 0, -1, 0};

    Node merge(Node a, Node b) {
        if (a.freq == 0) return b;
        if (b.freq == 0) return a;
        Node res;
        // prefix
        res.prefixVal = a.prefixVal;
        res.prefixFreq = a.prefixFreq;
        if (a.prefixVal == b.prefixVal) {
            res.prefixFreq += b.prefixFreq;
        }
        // suffix
        res.suffixVal = b.suffixVal;
        res.suffixFreq = b.suffixFreq;
        if (a.suffixVal == b.suffixVal) {
            res.suffixFreq += a.suffixFreq;
        }
        // default best
        res.val = a.val;
        res.freq = a.freq;
        if (b.freq > res.freq) {
            res.val = b.val;
            res.freq = b.freq;
        }
        // cross
        if (a.suffixVal == b.prefixVal) {
            int midFreq = a.suffixFreq + b.prefixFreq;
            if (midFreq > res.freq) {
                res.val = a.suffixVal;
                res.freq = midFreq;
            }
        }
    }
};
```

```

    }
    return res;
}

SegmentTree(const vector<int>& input) {
    int n = input.size();
    sz = 1;
    while (sz < n) sz *= 2;
    arr = input;
    tree.resize(sz * 2, skip);
    build(0, sz - 1, 0);
}

void build(int l, int r, int node) {
    if (l == r) {
        if (l < (int)arr.size()) {
            int v = arr[l];
            tree[node] = {v, 1, v, 1, v, 1};
        }
        return;
    }

    build(l, mid, L);
    build(mid + 1, r, R);
    tree[node] = merge(tree[L], tree[R]);
}

void update(int l, int r, int node, int idx, int val) {
    if (l == r) {
        arr[idx] = val;
        tree[node] = {val, 1, val, 1, val, 1};
        return;
    }

    if (idx <= mid)
        update(l, mid, L, idx, val);
    else
        update(mid + 1, r, R, idx, val);

    tree[node] = merge(tree[L], tree[R]);
}

Node query(int l, int r, int node, int ql, int qr) {
    if (r < ql || l > qr) return skip; // neutral
    if (ql <= l && r <= qr) return tree[node];
    Node left = query(l, mid, L, ql, qr);
    Node right = query(mid + 1, r, R, ql, qr);
    return merge(left, right);
}

void updateCall(int idx, int val) {
    update(0, sz - 1, 0, idx, val);
}

Node queryCall(int l, int r) {

```



```

        return query(0, sz - 1, 0, l, r);
    }

    #undef L
    #undef R
    #undef mid
};

```

Segment Tree with pointers :

```

const int skip = 0;
struct Node
{
    int val ;
    Node* lf , *ri ;
    Node (int v = skip){
        val = v;
        lf = ri = nullptr;
    }
};
struct SegTreePtr
{
    Node* root = nullptr;
    Node* update(int ind , int val , Node* node , int l = 0 , int r = 1e9+5){
        if(!node) node = new Node();
        if(l == r){
            node->val += val ;
            return node;
        }
        int mid = (l + r) / 2;
        if(ind <= mid)
            node->lf = update(ind , val , node->lf , l , mid);
        else
            node->ri = update(ind , val , node->ri , mid+1 , r);

        int left = node->lf ? node->lf->val : skip ;
        int right = node->ri ? node->ri->val : skip ;

        node->val = left + right;
        return node ;
    }
    int query (int lx , int rx , Node*& node , int l = 0 , int r = 1e9+5){
        if( !node || lx > r || l > rx ) return skip ;
        if(lx <= l && rx >= r) return node->val ;

        int mid = (l + r) / 2;

        return query(lx , rx , node->lf , l , mid) +
            query(lx , rx , node->ri , mid+1 , r);
    }
};

```

```
    }
};
```

Merge Sort Tree :

```
struct MergeSortTree {
    int n;
    vector<vector<int>> seg;

    MergeSortTree(vector<int> &a) {
        n = a.size();
        seg.resize(4 * n);
        build(1, 0, n - 1, a);
    }

    void build(int node, int l, int r, vector<int> &a) {
        if (l == r) {
            seg[node] = {a[l]};
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid, a);
        build(2 * node + 1, mid + 1, r, a);

        merge(seg[2 * node].begin(), seg[2 * node].end(),
              seg[2 * node + 1].begin(), seg[2 * node + 1].end(),
              back_inserter(seg[node]));
    }

    int query(int node, int l, int r, int ql, int qr, int x) {
        if (r < ql || l > qr) return 0;

        if (ql <= l && r <= qr) {
            auto &v = seg[node];
            return v.end() - upper_bound(v.begin(), v.end(), x);
        }

        int mid = (l + r) / 2;
        return query(2 * node, l, mid, ql, qr, x) +
               query(2 * node + 1, mid + 1, r, ql, qr, x);
    }

    int query(int l, int r, int x) {
        return query(1, 0, n - 1, l, r, x);
    }
};
```

Trie

```
struct Node
{
    map<char,int>mp ;
    int sz = 0 , isEnd = 0;
    int &operator[](char x){
        return mp[x];
    }
};

struct Trie
{
    vector<Node>tr ;
    int newNode (){
        tr.emplace_back();
        return tr.size()-1 ;
    }
    Trie(){
        tr.clear() ;
        newNode() ;
    }
    void insert(string &s , int op){
        int cur = 0 ;
        for(auto ch : s){
            if(!tr[cur][ch]){
                tr[cur][ch] = newNode();
            }
            cur = tr[cur][ch];
            tr[cur].sz += op ;
        }
        tr[cur].isEnd += op ;
    }

    int search (string &s){
        int cur = 0 ;
        for(auto ch : s){
            if(!tr[cur][ch]){
                return 0 ;
            }
            cur = tr[cur][ch];
        }
        return tr[cur].sz ;
    }
};
```

Binary Trie

```

int M = 60 ;

struct Node
{
    int mp[2] = {0,0} ;
    int sz = 0 ;
    int isEnd = 0 ;
    int &operator[](int x){
        return mp[x] ;
    };
};

struct Trie
{
    vector<Node>tr ;
    int newNode(){
        tr.emplace_back() ;
        return tr.size()-1 ;
    }
    Trie(){
        tr.clear() ;
        newNode();
    }
    void insert(int x , int op){
        int cur = 0 ;
        for (int i = M-1; i >= 0; i--)
        {
            int u = x>>i & 111 ;
            if(tr[cur][u]==0){
                tr[cur][u] = newNode();
            }
            cur = tr[cur][u] ;
            tr[cur].sz+=op ;
        }
        tr[cur].isEnd+=op ;
    }
    // min val of a[i] ^ x
    int queryMin(int x){
        int cur = 0 ; int res = 0 ;
        for (int i = M-1; i >= 0; i--)
        {
            int u = x>>i & 111 ;
            int v = tr[cur][u];
            if(!tr[v].sz){
                res+=(111<<i) ;
                cur = tr[cur][!u];
            }else{
                cur = v ;
            }
        }
        return res ;
    }
    // max val of a[i] ^ x

```

```

int queryMax(int x) {
    int cur = 0 ; int res = 0 ;
    for (int i = M - 1; i >= 0; --i) {
        int u = x >> i & 111;
        int v = tr[cur][!u];
        if(tr[v].sz){
            res+=(111<<i) ;
            cur = v;
        }else{
            cur = tr[cur][u] ;
        }
    }
    return res ;
}

// count how many times x is in the trie
int count(int x){
    int cur = 0 ;
    for (int i = M-1; i >= 0; i--){
        int u = x>>i & 111 ;
        if(tr[cur][u]==0) return 0 ;
        cur = tr[cur][u] ;
    }
    return tr[cur].isEnd ;
}

// count how many numbers in trie are < x
int countLess(int x){
    int cur = 0 ; int res = 0 ;
    for (int i = M-1; i >= 0; i--){
        int u = x>>i & 111 ;
        if(u==1 && tr[cur][0]) res += tr[tr[cur][0]].sz ;
        if(tr[cur][u]==0) break ;
        cur = tr[cur][u] ;
    }
    return res ;
}

// count number of a[i]^num <= K
int query(int num, int K){
    int cur = 0 , res = 0 ;
    for(int i = M - 1 ; i >= 0 ; i--){
        int n = num >> i & 111 ;
        int k = K >> i & 111 ;
        if(n == 0 && k == 0){
            cur = tr[cur][0] ;
        }else if(n == 0 && k == 1){
            if(tr[cur][0]) res += tr[tr[cur][0]].sz ;
            cur = tr[cur][1] ;
        }else if(n == 1 && k == 0){
            cur = tr[cur][1] ;
        }else{ // n == 1 && k == 1
            if(tr[cur][1]) res += tr[tr[cur][1]].sz ;
            cur = tr[cur][0] ;
        }
    }
    if(cur == 0) break ;
}

```

```

    if(cur) res += tr[cur].isEnd ;
    return res ;
}

int kth(int k){
    int cur = 0, res = 0;
    for(int i = M - 1; i >= 0; i--){
        int left = tr[cur][0];
        int szLeft = (left ? tr[left].sz : 0);
        if(k <= szLeft){
            cur = left;
        }else{
            k -= szLeft;
            res |= (1ll << i);
            cur = tr[cur][1];
        }
        if(cur == 0) break;
    }
    return res;
}

};

```

Sqrt Decomposition

```

int n , q ; cin>>n>>q;
int arr[n];
int BQ = sqrt(n)+2 ;
vi sq (BQ) ;
lp(i,n)cin>>arr[i] , sq[i/BQ]+= arr[i];

auto update = [&](int i , int x){
    int bn = i/BQ ;
    sq[bn]-= arr[i];
    arr[i] = x ;
    sq[bn]+= arr[i];
};

auto query = [&](int r){
    int bn = r/BQ , ans = 0 ;
    for (int i = 0; i < bn; i++)
    {
        ans += sq[i];
    }
    for (int i = bn*BQ; i < r; i++)
    {
        ans += arr[i];
    }
}

```

```

        return ans ;
    };

```

Persistent Segment tree

```

struct Node
{
    Node *l, *r;
    int s;

    Node(int s = 0, Node* l = NULL, Node* r = NULL)
        : s(s), l(l), r(r) {}
};

Node* newLeaf(int val){return new Node(val);}

Node* newParent(Node* l, Node* r){return new Node(l->s + r->s, l, r);}

Node* build(vector<int>& arr, int l, int r)
{
    if (l == r)
        return newLeaf(arr[l]);

    int m = (l + r) / 2;

    return newParent(
        build(arr, l, m),
        build(arr, m + 1, r)
    );
}

int query(Node* node, int l, int r, int lx, int rx)
{
    if (r < lx || l > rx) return 0;

    if (lx <= l && r <= rx) return node->s;

    int m = (l + r) / 2;

    return query(node->l, l, m, lx, rx) +
        query(node->r, m + 1, r, lx, rx);
}

Node* update(Node* node, int l, int r, int idx, int val)
{
    if (l == r)
        return newLeaf(val);

    int m = (l + r) / 2;

```

```

    if (idx <= m){
        return newParent(update(node->l, l, m, idx, val),node->r);
    }
    else{
        return newParent(node->l,update(node->r, m + 1, r, idx, val));
    }
}

```

Dynamic Persistent Segment tree :

```

const int N = 1e9+7;
struct Node
{
    Node *l, *r;
    int s;

    Node(int s = 0, Node* l = NULL, Node* r = NULL)
        : s(s), l(l), r(r) {}
};

Node* newLeaf(int val){return new Node(val);}

Node* newParent(Node* l, Node* r){
    int sum = 0;
    if (l) sum += l->s;
    if (r) sum += r->s;
    return new Node(sum, l, r);
}

int query(Node* node, int l, int r, int lx, int rx)
{
    if (!node || r < lx || l > rx)return 0;

    if (lx <= l && r <= rx)return node->s;

    int m = (l + r) / 2;

    return query(node->l, l, m, lx, rx) +
           query(node->r, m + 1, r, lx, rx);
}

Node* update(Node* node, int l, int r, int idx, int val)
{
    if (!node)
        node = new Node();

    if (l == r)
        return newLeaf(val + node->s);
}

```



```

int m = (l + r) / 2;

if (idx <= m){
    return newParent(update(node->l, l, m, idx, val), node->r);
}
else{
    return newParent(node->l, update(node->r, m + 1, r, idx, val));
}
}

```

Dynamic Lazy Segment tree:

```

struct Node {
    ll val = 0 , lazy = 0 , typ = -1;
    Node *left = NULL;
    Node *right = NULL;
    void update(int x, int lx, int rx) {
        val += 1ll * x * ((rx - lx + 1));
        lazy += x, typ = 1;
    }
};

void push(Node *node, ll l, ll r) {

    if(l == r) return ;
    if (!node->left) node->left = new Node();
    if (!node->right) node->right = new Node();

    if(node->typ == -1) return ;

    ll mid = (l + r) >> 1;
    node->left->update(node->lazy, l, mid);
    node->right->update(node->lazy, mid+1 , r);

    node->lazy = 0; node->typ = -1 ;

}

void update(Node *node, ll l, ll r, ll lx, ll rx, ll val) {

    if (rx < l || r < lx) return;
    if (lx <= l && r <= rx) return node->update(val , l , r);

    push(node , l , r);

    ll mid = (l + r) >> 1;

    update(node->left, l, mid, lx, rx, val);

```

```

    update(node->right, mid + 1, r, lx, rx, val);
    node->val = (node->left->val + node->right->val); // merge
}

ll query(Node *node, ll l, ll r, ll lx, ll rx) {

    if (rx < l || r < lx) return 0;
    if (lx <= l && r <= rx) return node->val;
    push(node, l, r);

    ll mid = (l + r) >> 1;

    return (query(node->left, l, mid, lx, rx) +
            query(node->right, mid + 1, r, lx, rx)); // merge
}

```

Lazy Persistent Segment Tree :

```

struct Node {
    ll sum, lazy;
    Node *l, *r;
    Node() {
        sum = lazy = 0;
        l = r = nullptr;
    }

    Node(const Node &o) {
        sum = o.sum;
        lazy = o.lazy;
        l = o.l;
        r = o.r;
    }
};

int N, M;
ll A[MAXN];

Node* build(int l, int r) {
    Node *node = new Node();

    if (l == r) {
        node->sum = A[l];
        return node;
    }

    int mid = (l + r) >> 1;

    node->l = build(l, mid);
    node->r = build(mid + 1, r);
}

```

```

    node->sum = node->l->sum + node->r->sum;

    return node;
}

Node* update(Node *node, int l, int r, int ql, int qr, ll val) {

    if (qr < l || r < ql) return node;

    Node *cur = new Node(*node);

    if (ql <= l && r <= qr) {
        cur->sum += val * (r - l + 1);
        cur->lazy += val;
        return cur;
    }

    int mid = (l + r) >> 1;

    cur->l = update(cur->l, l, mid, ql, qr, val);
    cur->r = update(cur->r, mid + 1, r, ql, qr, val);

    cur->sum = cur->l->sum + cur->r->sum + cur->lazy * (r - l + 1);

    return cur;
}

ll query(Node *node, int l, int r, int ql, int qr, ll carry = 0) {

    if (qr < l || r < ql) return 0;

    if (ql <= l && r <= qr) return node->sum + carry * (r - l + 1);

    carry += node->lazy;

    int mid = (l + r) >> 1;

    return query(node->l, l, mid, ql, qr, carry) + query(node->r, mid + 1, r, ql,
qr, carry);
}

```

segment Tree polynomial query:

```

class Node {
public:
    long long sum;
    long long wsum;

    Node() {

```

```

        sum = 0;
        wsum = 0;
    }

    Node(long long data) {
        sum = data;
        wsum = data; // For a single element, length is 1, so wsum = data * 1
    }

    Node(long long s, long long ws) {
        sum = s;
        wsum = ws;
    }

    static Node merge(Node left, Node right, long long left_len) {
        long long s = left.sum + right.sum;
        long long ws = left.wsum + right.wsum + (right.sum * left_len);
        return Node(s, ws);
    }
};

class Segtree {
#define L 2*node+1
#define R 2*node+2
#define mid (l + (r-l)/2)
#define ll long long
public:
    vector<Node> seg;
    Node skip;
    ll sz;

    Segtree(vector<ll> &v){
        sz = 1;
        while (sz < v.size()) sz *= 2;
        seg.assign(2*sz, skip);
        build(v, 0, sz-1, 0);
    }

    void build(vector<ll> &v, ll l, ll r, ll node) {
        if (l == r) {
            if (l < v.size())
                seg[node] = Node(v[l]);
            return;
        }

        build(v, l, mid, L);
        build(v, mid+1, r, R);
        ll left_len = (mid - l + 1);
        seg[node] = Node::merge(seg[L], seg[R], left_len);
    }

    void update(ll l, ll r, ll node, ll idx, ll val) {
        if (l == r) {
            seg[node] = Node(val);

```

```

        return;
    }
    if (idx <= mid)
        update(l, mid, L, idx, val);
    else
        update(mid+1, r, R, idx, val);

    ll left_len = (mid - l + 1);
    seg[node] = Node::merge(seg[L], seg[R], left_len);
}

void update(ll idx, ll val) {
    update(0, sz-1, 0, idx, val);
}

Node query(ll l, ll r, ll node, ll lx, ll rx) {
    if (r < lx || rx < l) return Node(0, 0); // Identity element for sum/wsum
    if (l >= lx && r <= rx) return seg[node];

    ll mid_val = mid;
    Node left = query(l, mid_val, L, lx, rx);
    Node right = query(mid+1, r, R, lx, rx);

    // The effective length of the 'left' part participating in the query
    overlap
    ll left_len = max(0LL, min(rx, mid_val) - max(l, lx) + 1);
    return Node::merge(left, right, left_len);
}

ll query(ll l, ll r) {
    // Returns the weighted sum query for range [l, r]
    return query(0, sz-1, 0, l, r).wsum;
}

#undef L
#undef R
#undef mid
#undef ll
};

```

Poly Queries (update) :

```

#include <bits/stdc++.h>
using namespace std;

#define int long long

const int N = 200005;

```

```
int n, q;
int seg[4 * N];
int lazyStart[4 * N];
int lazyDiff[4 * N];

void apply(int node, int l, int r, int start, int diff) {
    int len = r - l + 1;
    seg[node] += (2 * start + (len - 1) * diff) * len / 2;
    lazyStart[node] += start;
    lazyDiff[node] += diff;
}

void push(int node, int l, int r) {
    if (lazyStart[node] == 0 && lazyDiff[node] == 0) return;

    int mid = (l + r) / 2;
    int leftLen = mid - l + 1;

    apply(node * 2, l, mid,
          lazyStart[node],
          lazyDiff[node]);

    apply(node * 2 + 1, mid + 1, r,
          lazyStart[node] + leftLen * lazyDiff[node],
          lazyDiff[node]);

    lazyStart[node] = lazyDiff[node] = 0;
}

void build(int node, int l, int r) {
    if (l == r) {
        cin >> seg[node];
        return;
    }

    int mid = (l + r) / 2;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);

    seg[node] = seg[node * 2] + seg[node * 2 + 1];
}

void update(int node, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return;

    if (ql <= l && r <= qr) {
        apply(node, l, r, l - ql + 1, 1);
        return;
    }

    push(node, l, r);

    int mid = (l + r) / 2;
    update(node * 2, l, mid, ql, qr);
```

```
    update(node * 2 + 1, mid + 1, r, ql, qr);

    seg[node] = seg[node * 2] + seg[node * 2 + 1];
}

int query(int node, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;

    if (ql <= l && r <= qr) return seg[node];

    push(node, l, r);

    int mid = (l + r) / 2;
    return query(node * 2, l, mid, ql, qr)
        + query(node * 2 + 1, mid + 1, r, ql, qr);
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> q;
    build(1, 1, n);

    while (q--) {
        int t, l, r;
        cin >> t >> l >> r;

        if (t == 1)
            update(1, 1, n, l, r);
        else
            cout << query(1, 1, n, l, r) << '\n';
    }
}
```